Cardinal Court

Voice Receptionist — Solution Plan & Delivery

A two-track plan for the LiveKit voice-receptionist exercise. Version 1 is the full submission build delivered inside the hard 3-hour cap — planning, credentials, build, deployment, testing, and a small eval pass. Version 2 collects the brief's optional stretch goals for a more polished build after submission. Scope is bounded strictly by the take-home brief and the building fact pack; current LiveKit specifics are verified against the live documentation.

Source documents	Take-home brief (Build a Voice Receptionist with LiveKit) · building-fact-pack-cardinal-court
Deliverable (brief)	A LiveKit voice agent at a public URL + a public GitHub repo + a short README
V1 stack	Python · LiveKit Agents 1.x (AgentSession) · LiveKit Cloud (managed deploy + Inference)
V1 grounding	In-context structured KB from the fact pack (grounding method is the candidate's call)
V1 time box	Aim 2–3 hours · hard cap 3 hours
V2	Brief's named stretch goals only (MCP, RAG + eval method, telephony, multi-language, traces)

1 · Objective and source boundary

Per the brief, the agent must: be a real LiveKit voice agent (speech-in / speech-out, not a text chatbot); be reachable from a public URL; know Cardinal Court and answer building questions accurately from the fact pack; behave like a receptionist and not invent answers (including saying so for a company that isn't in the building or for something it doesn't have, such as a tenant's direct number); and live on public GitHub with a short README covering how to run it, what was used, what you'd improve, and roughly where time went.

Handback (brief): the live URL, the GitHub repo URL, and a one-line note before dialling in.

Source boundary for this document

Everything planned here traces to one of: the take-home brief (core requirement or named stretch goal), the building fact pack, the brief's assessment rubric, or verified current LiveKit documentation. The traceability matrix in Section 12 lists each item against its source. Nothing outside those sources is treated as planned work.

Governing principle from the rubric

The brief weighs engineering judgement and scoping inside the time box, and states plainly that a simple, working, well-explained agent beats an ambitious broken one, that stretch goals are "conversation fuel, not requirements," and that it does not score visual polish, test coverage, or completeness of stretch goals. V1 is therefore kept deliberately lean; V2 is explicitly optional.

2 · Solution architecture (V1)

LiveKit Agents 1.x centres on AgentSession, which the SDK uses to stream audio through the STT → LLM → TTS pipeline and to handle turn detection and interruption. The agent worker joins a LiveKit room as a realtime participant; the public URL is served by a separate frontend that mints a token and joins the same room.

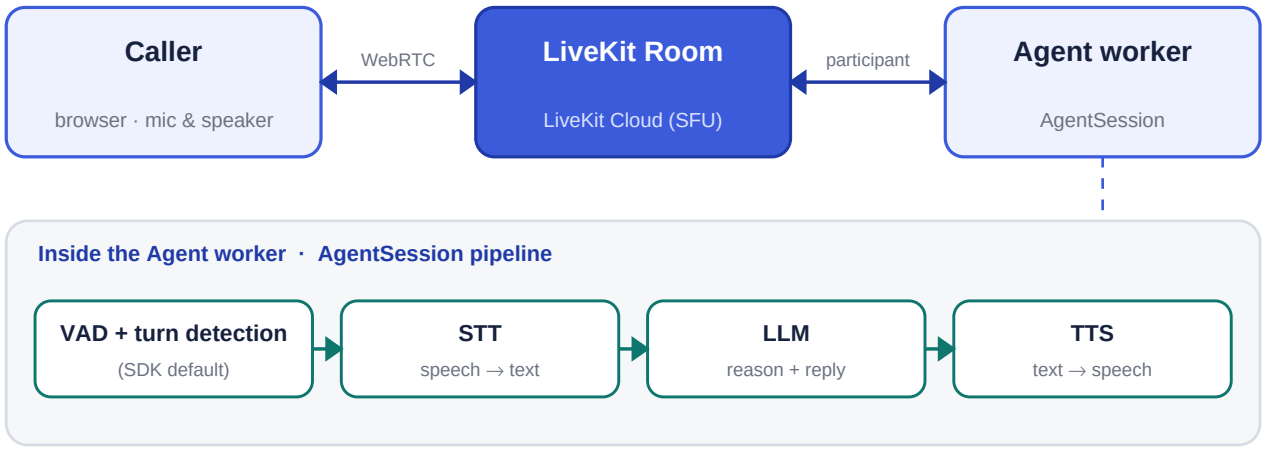

Figure 1. The realtime audio path. The public URL is served by a separate frontend that joins the same room; the agent worker runs on LiveKit Cloud (managed) or your own host via the starter's Dockerfile.

Barge-in is effectively free in V1

The brief lists "graceful barge-in (let the caller interrupt)" as a stretch goal. In LiveKit Agents 1.x the SDK plus the LiveKit turn detector handle interruption by default, so V1 already supports it — it belongs to V1, not V2.

Grounding (the candidate's call, per the brief)

V1 grounds in-context: the fact pack is held as a structured knowledge file and placed in the system prompt. The corpus is small, so this keeps latency low (no per-turn retrieval round trip), avoids retrieval failures on questions that need two facts at once, and is the least to build. Retrieval (RAG) and an MCP tool are deferred to V2, where the brief lists them as stretch goals.

3 · Prerequisites and credentials (API keys)

The brief calls for a LiveKit project (free tier) plus STT/LLM/TTS providers on free credit, and says to email Nathan for a LiveKit sandbox if keys or credits become a blocker. Two routes:

Credential	Where it comes from	Used as
LIVEKIT_URL	LiveKit Cloud project (free Build tier)	.env.local + deploy/CI secret
LIVEKIT_API_KEY	LiveKit Cloud project settings	.env.local + deploy/CI secret
LIVEKIT_API_SECRET	LiveKit Cloud project settings	.env.local + deploy/CI secret
Model access	Route A — LiveKit Inference (no separate provider account needed); Route B — provider keys: Deepgram/AssemblyAI (STT), OpenAI/Anthropic (LLM), Cartesia/ElevenLabs (TTS) on free credit	.env.local
Frontend token	LiveKit sandbox token server, or the frontend's own API route	frontend env

Route A (LiveKit Inference) is the faster path for the time box because it accesses STT/LLM/TTS through the LiveKit Cloud credentials, removing separate provider sign-ups. Route B (plugins with direct keys) is available if a specific provider is preferred. Either way, keep everything on free tiers as the brief instructs.

```
# .env.local (local + copied into deploy/CI secrets)
LIVEKIT_URL=wss://<project>.livekit.cloud
LIVEKIT_API_KEY=...
LIVEKIT_API_SECRET=...
# Route B only (if using provider plugins instead of Inference):
# DEEPGRAM_API_KEY=...   OPENAI_API_KEY=...   CARTESIA_API_KEY=...
```

Fallback if credentials block you

The brief explicitly offers a LiveKit sandbox if keys or credits are a blocker, and says that if persistent hosting is awkward it can be arranged to run together at the start of the session. Don't lose time fighting billing.

4 · Build and grounding

Scaffold from the official Python starter (it ships the turn detector, a Dockerfile, and a pytest eval suite), then add the building knowledge and the receptionist behaviour.

```
lk cloud auth
lk agent init cardinal-court --template agent-starter-python
# add knowledge-base.yaml; load it into the system prompt in the agent
# download required models (e.g. Silero VAD + turn detector) before first run
# run locally in console/terminal mode, then test in the Agents Playground
```

Behaviour to get right (from the fact pack and the brief's core):

- Accurate lookups from the fact pack: floors, tenants, amenities, access, transport, hours.
- Multi-fact answers: a courier with a parcel for a tenant goes to the lower-ground parcel room, not to the tenant's floor.
- Refusals without invention: a company not in the building, a question with no data, and a tenant's direct number.
- Misheard tenant names confirmed before directing (phone ASR mangles names like Loom, Kiln, Meridian).
- Details a sloppy agent drops: Loom on floors 3 and 4 (desk on 3); ID-required tenants; café hours vs building hours.
- Concise, spoken-style answers; greets like a front desk.

5 · Deployment

- **Agent worker** → LiveKit Cloud (managed deployment, no code changes), or any host using the starter's Dockerfile.
- **Frontend** → deploy the LiveKit React starter (e.g. to Vercel); set the company name to Cardinal Court (a one-minute change — the rubric does not score visual polish).
- **Public URL** = the deployed frontend; confirm a full spoken conversation works through it, not just locally.
- **Persistence** for the interview slot: keep it up, or use the sandbox path; the brief allows arranging to run it together if hosting is awkward.

6 · Testing

The fact pack's own sample questions are the acceptance test — they are what will be asked in the live demo. The agent must pass the routine lookups and the awkward ones:

- Which floor is Loom on? · I've a meeting with Meridian Capital, where do I go? · Is it step-free?
- What time does the café open? · Where are the showers? · Can I park here? · Directions from London Bridge?
- Somewhere quiet to pray? · Courier with a parcel for Northwind Legal, where to? · Weekend opening hours?
- Can I book a meeting room? / Is the roof terrace open? · Medical emergency — is there a defibrillator?
- Awkward cases the brief calls out: a company not in the building, a question with no data, a private number.

Test through the Agents Playground and the hosted frontend; use headphones to avoid echo. Capture one conversation transcript/trace from LiveKit Cloud for the walkthrough.

7 · Evals (within the cap)

The brief lists "a tiny eval set of question → expected-answer you check against" as an optional stretch, and states it does not score test coverage. V1 therefore keeps evals small and honest: a handful of question→expected pairs drawn from the sample questions, including the awkward refusals, run through the starter's built-in testing/eval framework (pytest). A fuller suite is V2.

```
# evals/golden.yaml  (tiny: ~10-12 pairs from the sample questions)
- q: "Which floor is Loom on?"          expect_contains: ["3", "4"]
- q: "Parcel for Northwind Legal?"      expect_contains: ["parcel", "lower ground"]
- q: "Which floor is <unknown co>?"     expect_behavior: refuse
# run with: pytest   (starter's eval framework)
```

8 · The 3-hour timeline

Everything above sequenced inside the hard cap. If a phase overruns, stop and note what you'd do next — the brief rewards honest scoping over going over time.

Phase	Time	What happens
1. Plan + credentials	0:00–0:25	LiveKit Cloud project; keys (or Inference); scaffold the Python starter; quickstart talks locally.
2. Grounding + behaviour	0:25–1:10	Load the fact pack into the system prompt; write the receptionist persona and refusal guardrails.
3. Testing pass	1:10–1:40	Run the sample questions locally; fix multi-fact answers, refusals, and misheard names.
4. Deploy	1:40–2:20	Worker to LiveKit Cloud; frontend to Vercel → public URL; verify a full spoken conversation end to end.
5. Evals + trace	2:20–2:45	Tiny question→expected set via pytest; capture one conversation trace.
6. README + handback	2:45–3:00	README (run / used / improve / time spent); collect live URL, repo URL, and the one-line dial-in note.

9 · V1 definition of done and handback

- The fact pack's sample questions pass on the live URL, including the three awkward cases.
- Public GitHub repo with a README covering how to run it, what was used, what you'd improve, and where time went.
- Handback to Nathan: the live URL, the repo URL, and a one-line note (e.g. "give it a second to connect; best in Chrome").

10 · Version 2 — stretch goals (post-submission)

These are the brief's own optional stretch goals — "pick none, one, or several … conversation fuel, not requirements." They are built only after V1 is complete and submitted, each as an independent, shippable addition, ordered by signal. Nothing here is outside the brief's stretch list.

Stretch goal (from the brief)	What it adds	Note
MCP server	Expose the building data as an MCP server and have the agent call it as a tool, instead of baking the data into the prompt.	Highest signal; native SDK support.
RAG + evaluation method	A real retrieval step: chunk and embed the fact pack, retrieve on each question — plus how you'd evaluate whether retrieval is working (the brief asks for this explicitly).	Lets you justify the in-context choice by comparison.
Telephony (SIP)	Make it answerable on a real phone number via LiveKit SIP.	Free inbound number is US — note in README.
Multi-language	Handle non-English callers (multilingual STT + TTS).	Code-switching mid-sentence stays fragile.
Traces / fuller eval set	A trace/log of a few conversations, or a larger question→expected eval set.	Builds on V1's tiny set and captured trace.

Barge-in already delivered

The brief's other stretch goal — graceful barge-in — is handled by default in V1 (Section 2), so it is not repeated here.

11 · Deliberately not in scope

To respect the brief's scoping signals, effort is deliberately not spent on areas the rubric states are not scored:

- Visual polish beyond renaming the frontend — the rubric explicitly does not score visual polish.
- Broad automated test coverage — the rubric explicitly does not score test coverage; evals stay small.
- Completing every stretch goal — the rubric explicitly does not score stretch-goal completeness.

Anything not named in the brief or the fact pack is treated as a future idea for the walkthrough, not as planned work.

12 · Traceability matrix

Each planned element against its source, so nothing is unaccounted for.

Planned element	Track	Source
LiveKit voice agent (speech in/out)	V1	Brief — core
Reachable on a public URL	V1	Brief — core
Knows Cardinal Court (floors/tenants/amenities/access/transport/hours)	V1	Brief — core; Fact pack
Receptionist behaviour; no invention; refusals	V1	Brief — core

Planned element	Track	Source
Public GitHub + README (run/used/improve/time)	V1	Brief — core
In-context grounding choice	V1	Brief — grounding is candidate's call
Sample questions as acceptance test	V1	Fact pack — sample questions
Barge-in / turn detection	V1	Brief — stretch; LiveKit docs (default)
Tiny question→expected eval set + trace	V1	Brief — stretch (tiny eval / trace)
Stack, Inference, starter, deploy, SIP specifics	V1/V2	LiveKit docs (verified)
MCP server (data as a tool)	V2	Brief — stretch
RAG + how to evaluate retrieval	V2	Brief — stretch
Telephony (SIP phone number)	V2	Brief — stretch
Multi-language	V2	Brief — stretch
Fuller traces / eval set	V2	Brief — stretch

One-line summary

V1 delivers exactly the brief's core — a hosted LiveKit receptionist grounded in the fact pack, tested against the sample questions, with a small eval pass — inside 3 hours. V2 adds only the brief's named stretch goals, after submission, one at a time.